

Apache Kafka?

[Aman Arora](#)



If you are a software engineer working with back-end technologies or with big data, you probably would have heard about Apache Kafka by now — almost everybody is using it!

People using it are already going crazy about its features, capabilities and its superiority over other messaging systems. Terms like fault tolerance, replication, brokers, topics, zookeeper etc are quite common to be heard in regards to Kafka. These words seem hard at first but are actually quite easy to understand.

I am writing this post to help you understand why people are getting hooked to Kafka, and by the end, get you answers to the following questions -

What is Apache Kafka? Why people are using it? When should you use it? What are Producers, Consumers, Topics, Partitions, Brokers, Zookeeper?

Apache Kafka

Apache Kafka is an open source stream processing program that was

developed by LinkedIn but is now under the Apache foundation. It is written in Scala and Java.

Kafka can really help you create real-time, robust applications in conjunction with any programming language like Java, Node etc.

In Simpler terms, Apache Kafka is a Message broker i.e. it helps transmit messages from one system to another - in real time, reliable manner. But that's not all, Kafka can also work on streams of data and transform them (if required) using its stream API making it really helpful in a lot of use cases.

Now that we know what Apache Kafka is, let's move on to why it is so popular.

Why Kafka?

There are a lot of reasons for choosing Kafka as your message broker like replication, high performance, and fault tolerance. But what makes Kafka shine out are:

- Kafka's ability to scale without downtime
- Kafka's ability to work with high volumes of streams of data and provisions for transforming the aforementioned data, making it an ideal choice when working with big data

But before we dive down into the APIs that Kafka offer, it's important that we first understand the terminologies associated with Kafka.

Topics

Kafka stores streams of similar records in categories called Topics. The closest analogy of a topic would be a **table** in a relational database. Example: User location data for multiple users can be part of the same topic.

*Topics in Kafka are identified by the **topic name**.*

Partitions

Records in topics are further divided across partitions. A topic can have 'n' number of partitions. The number of partitions that a topic should have needs to be mentioned at the time of topic creation.

A large amount of thought needs to be put into deciding the number of partitions for a topic. Increasing the number of partitions for a topic improves the throughput but can also have some serious repercussions like — higher unavailability and an increased end-to-end latency.

A detailed description of how the number of partitions affects Kafka can be found [here](#).

<https://kafka.apache.org/intro>

In the above diagram, the *topic* has 3 partitions: Partition 0, Partition 1 and Partition 2. The numbers written inside each partition denote the *offset* of the message in that partition.

Offsets

Every message in a partition is assigned an integer value, called offset which uniquely identifies the message in the partition.

Within a partition a message with offset i is always processed before the message with offset $i+1$.

Kafka also stores the *write offset* for all the partitions for a topic, so that it knows where to insert the new record.

Producers

Producers publish data to the topic with the help of the topic name. It is the producer's responsibility to decide which message will go to which partition.

If there is no key associated with the topic the messages gets load balanced between the partitions round robin algorithm. But it is also possible to functionally determine (using some key in the message or some other variable factor) the partition to which a record should go.

Consumers

Compared to Producers, Consumers are a little tricky to understand. Consumers subscribe to topics and acts on messages. Consumers generally label themselves under a group — *consumer group*.

Messages that are published to a topic are sent to exactly one consumer instance within each subscribing consumer group i.e. consumers in a group never share a partition. This is to make sure that the same message doesn't get processed twice and also guarantees that the order of records processed for a partition.

If all the consumer instances belong to the same consumer group then the records are load balanced across all consumers.

Kafka fairly distributes the total number of partitions across the consumers in the consumer group.

For example — Let's say there is a topic which has 4 partitions(P_i) and there exists a consumer group (with 2 consumers) that is subscribed to this topic. Kafka now will try and divide the total partitions (4) across all the available consumers (2) i.e. each consumer will process the records of 2 partitions.

If there are more number of consumers than the number of partitions in a topic then one of the consumer will have to sit idle.

If any new consumer joins the group, some of the partitions are reassigned to the new instance. Similarly, if an instance dies, its partitions are reassigned to other consumers in the group.

Brokers

We use Kafka as a cluster, which means that there are generally one or more servers that work together to fulfil the requests. These servers are called Kafka brokers or brokers.

I hope you found this blog helpful and that I was able to transfer my knowledge of Kafka to you.

Checkout [Replication in Kafka](#) to find out how Kafka uses replication to overcome downtime in times of failure.

Thanks for reading...

You can follow me on LinkedIn by clicking [here](#)

Peace out!